



Алгоритмы обработки больших данных

L05. join, union, cogroup, windows, triggers, evictors
ProcessFunction, State, Timers, Watermarks

Апрель, 2026

Смирнов Сергей

smirnov.work@mail.ru

[@serg_v_smirnov](#)

Agenda

1. BigData intro
2. Stream & flink intro
3. Batch mode, pyflink runtime architecture
4. DataStream operators API, keyed state
- 5. *Stateful stream processing. Windows, Triggers, ProcessFunction, State, Timers, Watermarks***
6. State and Chekpoint, State Backends. Table API и SQL API
7. CEP, CDC, ML, Metrics, Failure recovery, Execution configuration, Async IO, Delivery guarantees. Best (worst) practices
8. Flink-SQL, базовый синтаксис, работа с окнами
9. Flink-SQL, соединение потоков, udf, оптимизация
10. Whats new, trends (fluss, streamhouse)

Homework



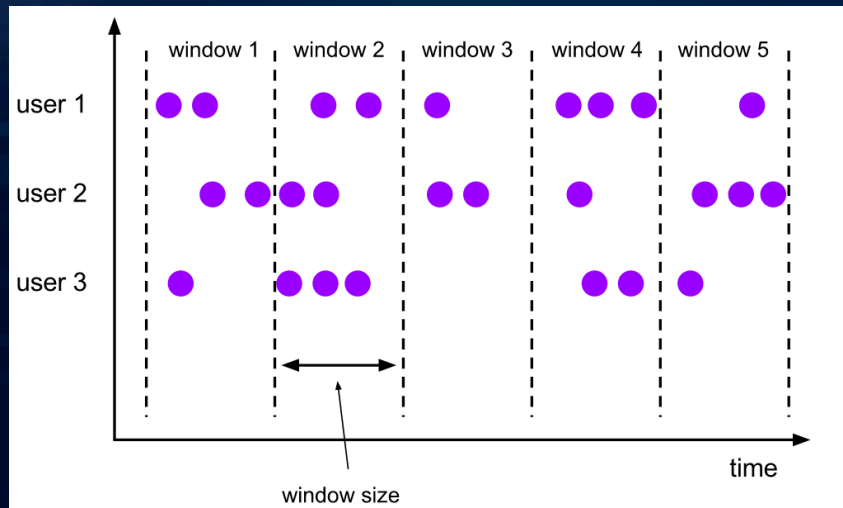
Доработать пример из предыдущего задания:

1. Поменять источник на кафку
2. Добавить оконные функции

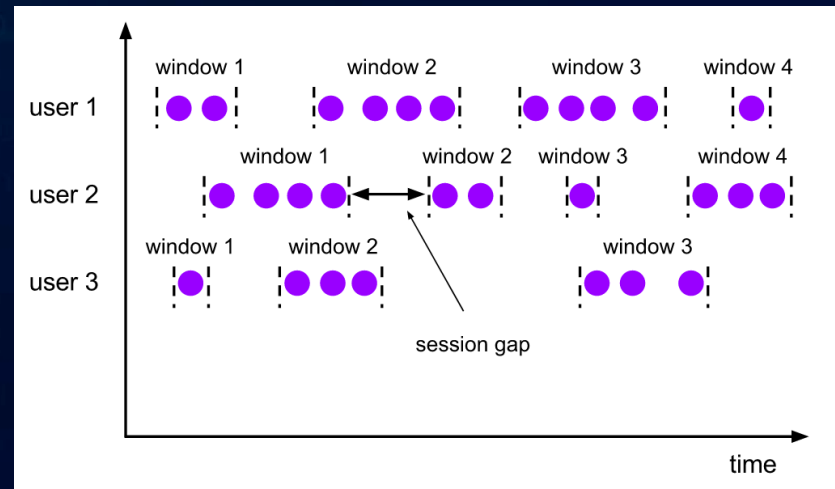
Запись результатов оставить в постгрес

Windows

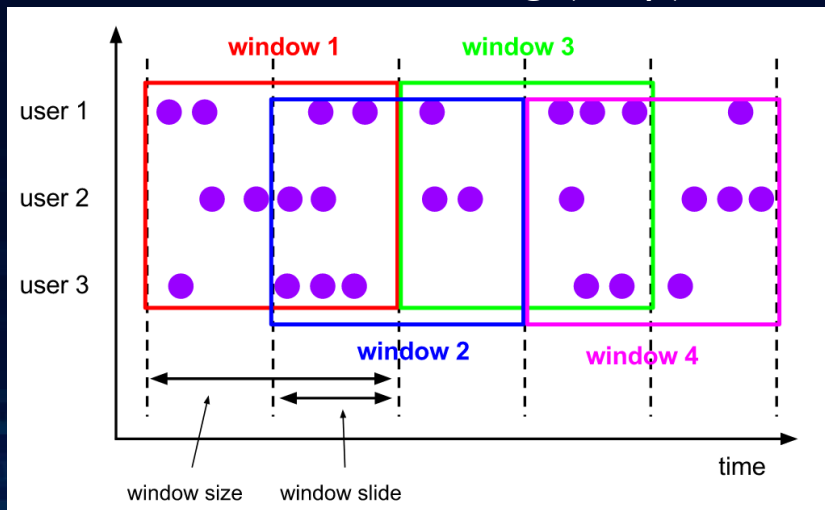
Tumbling Windows



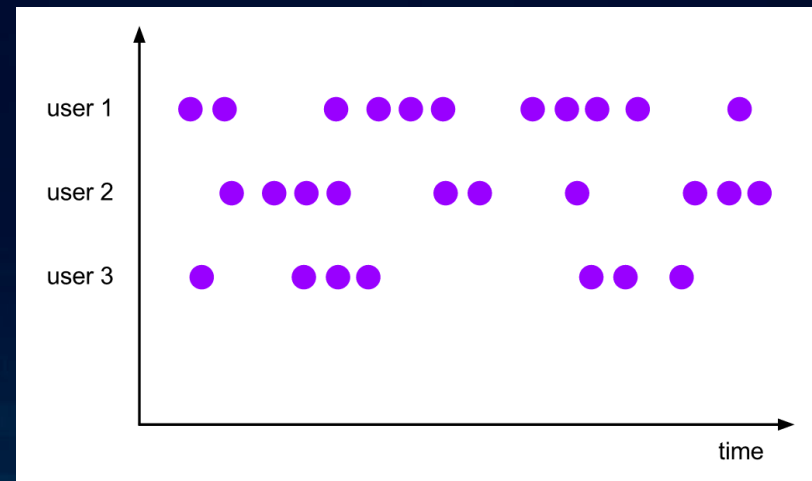
Session Windows



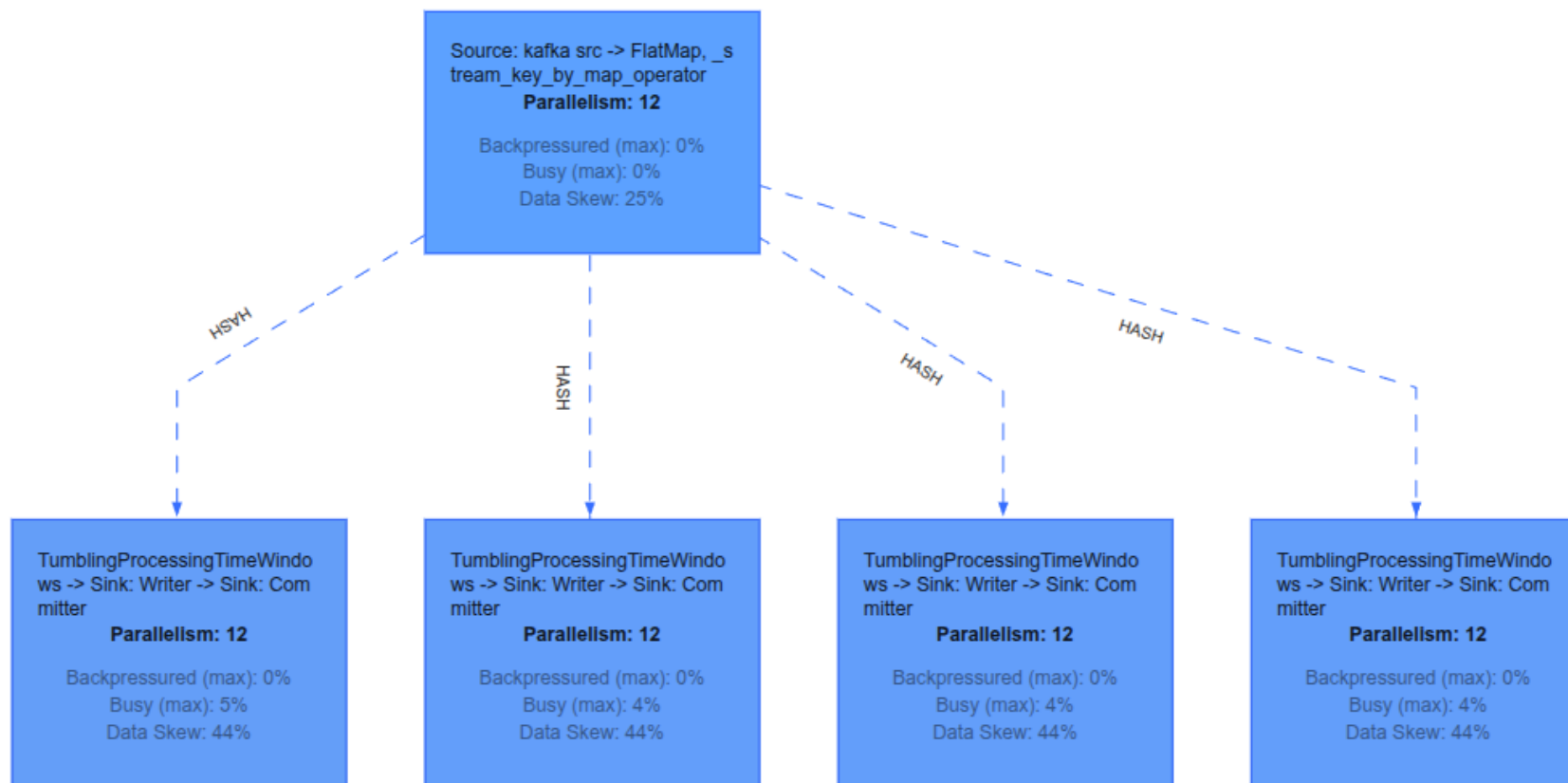
Sliding (Hop) Windows



Custom Windows



cnt, multiple implementation



WindowFunctions:

- ReduceFunction
- AggregateFunction
- WindowFunction
- ProcessWindowFunction

join, cogroup, connect

java specific

Join

```
.join().where().equalTo().window().apply()  
.intervalJoin().between().process()
```

Connect

common state
.flatMap1().flatMap2()

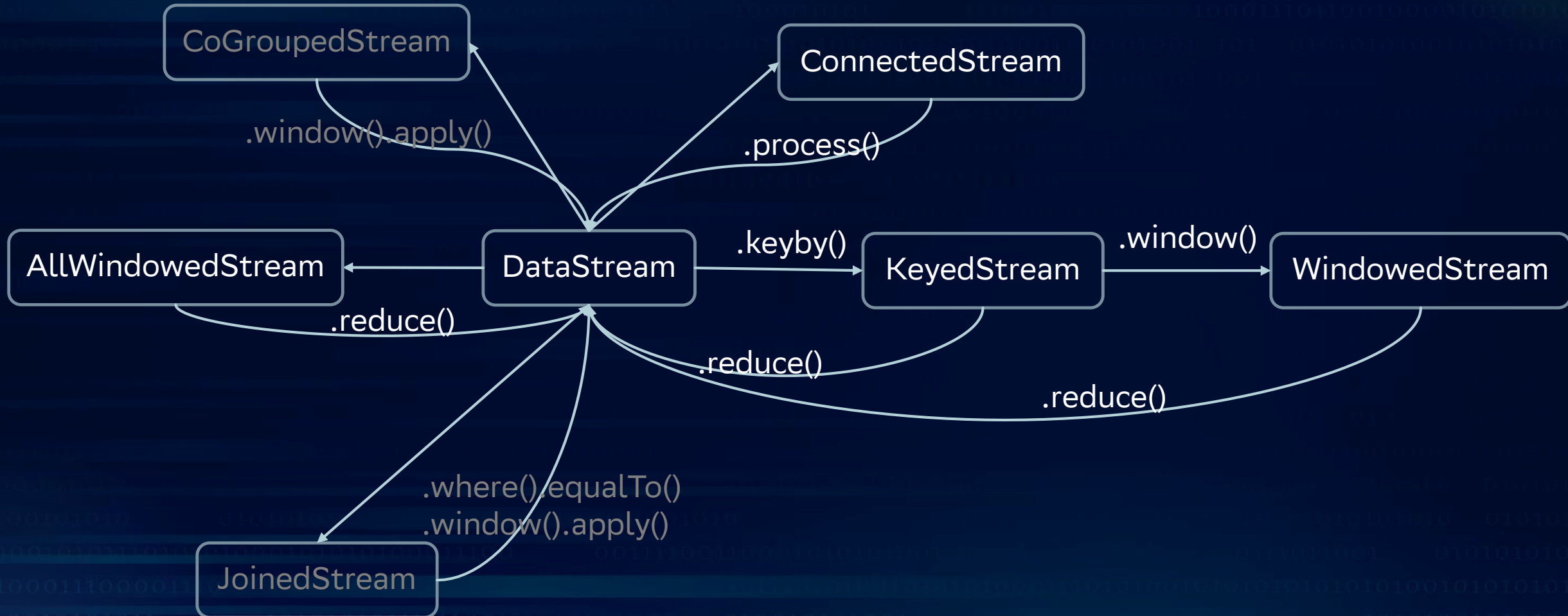
CoGroup

iterable

```
.join().where().equalTo().window().apply()
```

Stream transformation

java specific



window triggers

```
class CustomTrigger(Trigger):
    def __init__(self, limit):
        self.limit = limit
        self.count_state_desc = ValueStateDescriptor( name: "count", Types.INT())

    def on_element(self, element, timestamp, window, ctx):
        count_state = ctx.get_partitioned_state(self.count_state_desc)
        current_count = count_state.value()
        if current_count is None:
            current_count = 0
        current_count += 1
        count_state.update(current_count)
        if current_count >= self.limit:
            count_state.clear()
            return TriggerResult.FIRE_AND_PURGE
        return TriggerResult.CONTINUE

    def on_processing_time(self, time, window, ctx):
        return TriggerResult.CONTINUE

    def on_event_time(self, time, window, ctx):
        return TriggerResult.CONTINUE

    def on_merge(self, window, ctx: Trigger.OnMergeContext):
        return TriggerResult.CONTINUE

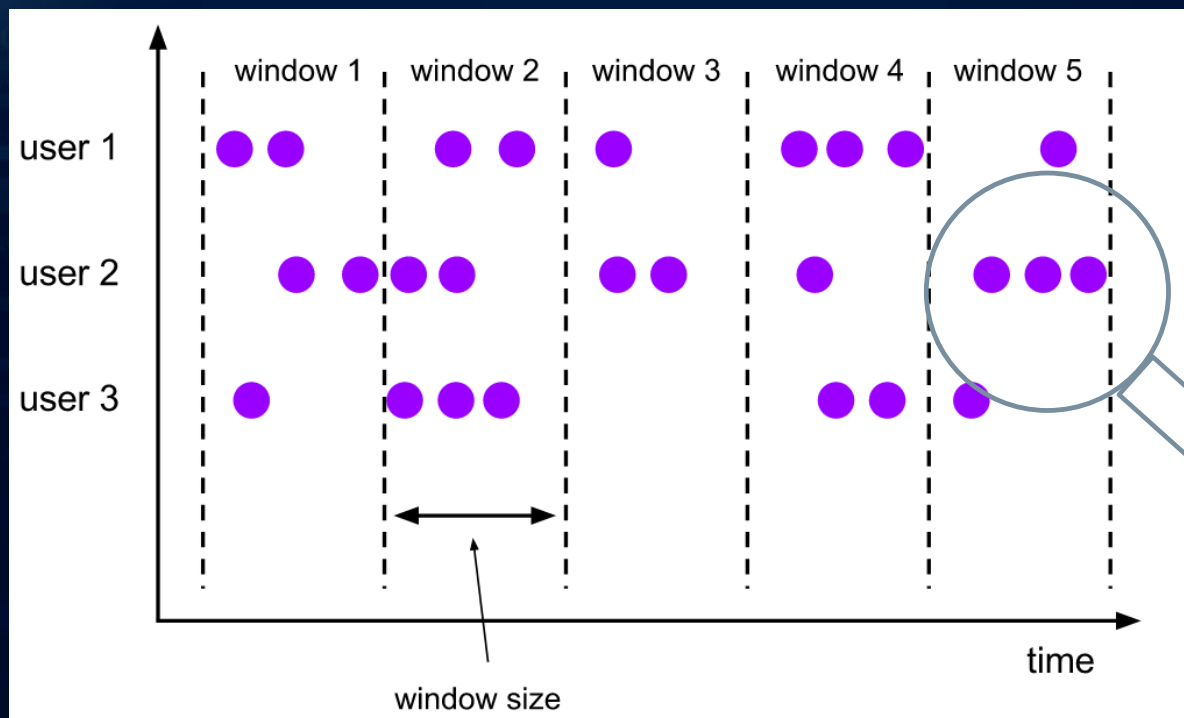
    def clear(self, window, ctx): 1 usage (1 dynamic)
        count_state = ctx.get_partitioned_state(self.count_state_desc)
        count_state.clear()
```

window evictors

java specific

```
.evictor(new Evictor<String, GlobalWindow>() {  
    @Override  
    public void evictBefore(Iterable<TimestampedValue<String>> elements, int size, GlobalWindow window, EvictorContext  
evictorContext) {  
  
    }  
  
    @Override  
    public void evictAfter(Iterable<TimestampedValue<String>> elements, int size, GlobalWindow window, EvictorContext  
evictorContext) {  
  
    }  
})
```

window, summarize



java specific

.trigger()

.evict()

-evictBefore

-evictAfter

CONTINUE
FIRE_AND_PURGE
FIRE
PURGE

продолжение работы
вызов оконной функции, очистка элементов
вызов оконной функции
очистка элементов

window, official site

java specific

Keyed Windows

Java/Scala

Python

```
stream
  .keyBy(...)          <- keyed versus non-keyed windows
  .window(...)         <- required: "assigner"
  [.trigger(...)]      <- optional: "trigger" (else default trigger)
  [.evictor(...)]       <- optional: "evictor" (else no evictor)
  [.allowedLateness(...)] <- optional: "lateness" (else zero)
  [.sideOutputLateData(...)] <- optional: "output tag" (else no side output for late data)
  .reduce/aggregate/apply() <- required: "function"
  [.getSideOutput(...)] <- optional: "output tag"
```

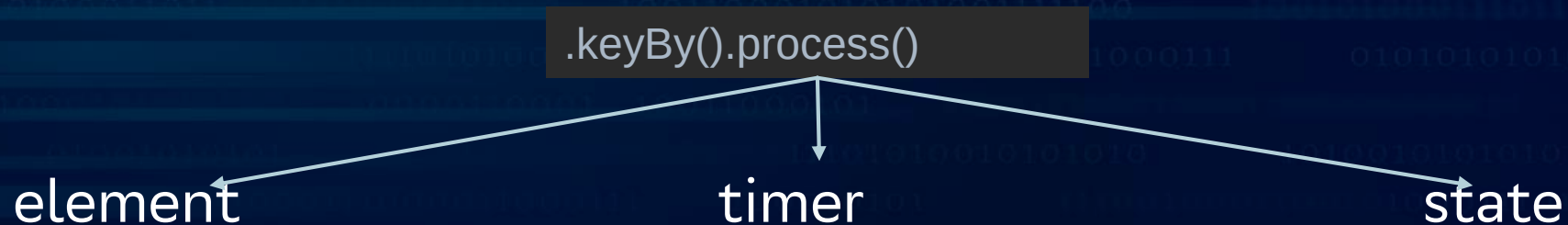
Non-Keyed Windows

Java/Scala

Python

```
stream
  .windowAll(...)      <- required: "assigner"
  [.trigger(...)]      <- optional: "trigger" (else default trigger)
  [.evictor(...)]       <- optional: "evictor" (else no evictor)
  [.allowedLateness(...)] <- optional: "lateness" (else zero)
  [.sideOutputLateData(...)] <- optional: "output tag" (else no side output for late data)
  .reduce/aggregate/apply() <- required: "function"
  [.getSideOutput(...)] <- optional: "output tag"
```

Process function



KeyedStream пререквизит для любой операции или функции с таймерами

State & timers поддерживают механизмы checkpoint и savepoint. Каждый зарегистрированный таймер, который должен был сработать (onTimer fired) вызывается сразу же после завершения восстановления

Регистрация таймера это и память и процессор. Служба TimeService дедублицирует таймеры по их ключам и времени срабатывания

Таймеры могут быть запущены и удалены

onTimer()



Processing time

Системные часы на основании которых (текущее время больше заданного) вызывается метод onTimer()



Event time

Собственные часы в виде watermarks при достижении которых (текущая величина watermark больше заданной) срабатывает метод onTimer()

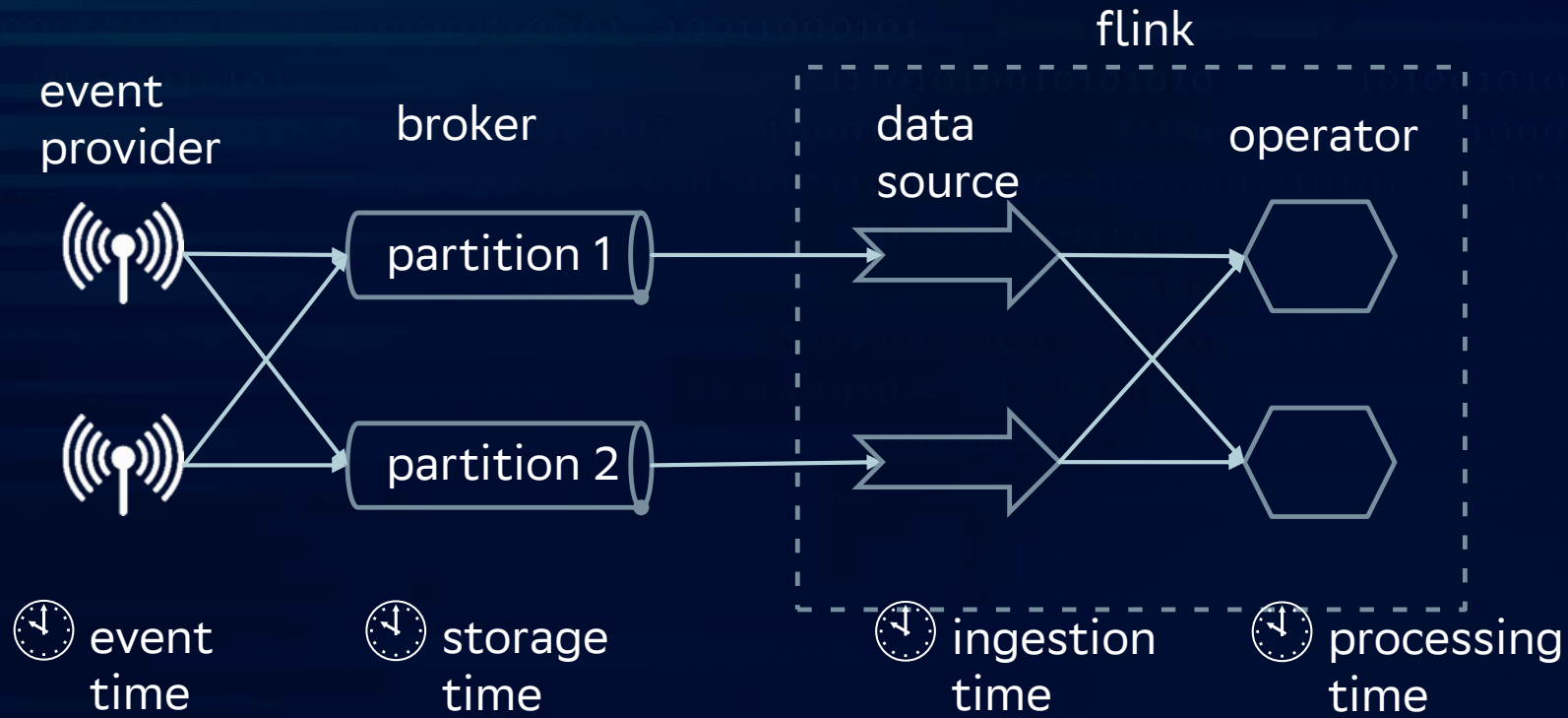
Во время вызова onTimer() передается контекст состояния в разрезе соответствующего ключа, согласно которому был создан таймер

Ключу на заданное время соответствует не более одного таймера

Взведенный заново на тоже время таймер уничтожает предыдущий

Доступ к состоянию по ключу по которому был создан таймер

Different time



В event time должна быть повторяемость результатов во времени

В processing time результат может меняться: сегодня один, завтра другой (недетерминированный результат)

Timers

Явное создание

```
ctx.timer_service().register_processing_time_timer(current_state.last_modified + 5000)
```

```
ctx.timer_service().register_event_time_timer(fire_at)
```

Неявное создание

```
.window(TumblingProcessingTimeWindows.of(Time.seconds(5)))
```

```
.window(TumblingEventTimeWindows.of(Time.seconds(5)))
```

State

```
class MyProcFunction(KeyedProcessFunction):
    def open(self, runtime_context: RuntimeContext):
        # Define the state descriptor
        state_descriptor = ValueStateDescriptor(
            "cnt",
            Types.PICKLED_BYTE_ARRAY() # used for custom python objects
        )
        # В объекте SimpleCnt хранится количество упоминаний слова и время последнего обновления
        # Данные сериализуются через PICKLED_BYTE_ARRAY (позволяет хранить произвольные объекты)
        self.state = runtime_context.get_state(state_descriptor)

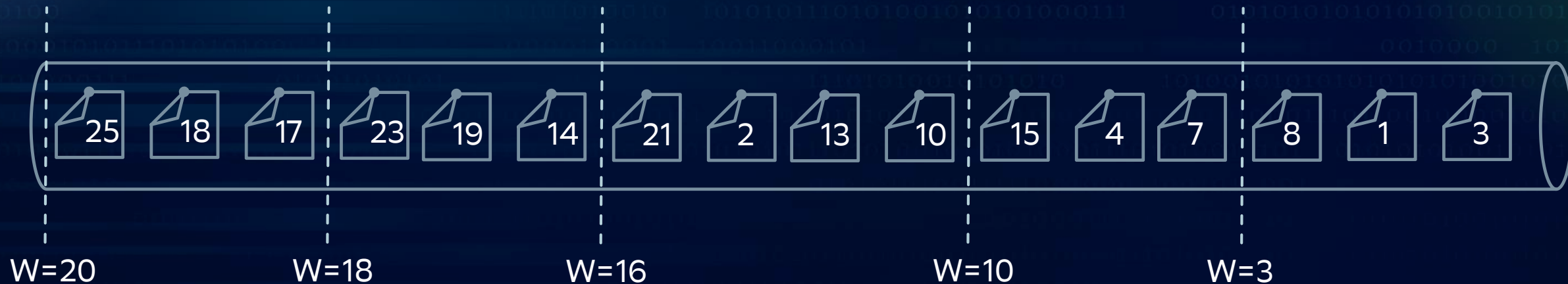
    def process_element(self, value, ctx: 'KeyedProcessFunction.Context'):
        current_state = self.state.value()
        if current_state is None:
            current_state = SimpleCnt(key=value)
            current_state.last_modified = ctx.timer_service().current_processing_time()
            # При получении нового слова регистрируется таймер на 5 секунд вперед (+ 5000).
            ctx.timer_service().register_processing_time_timer(current_state.last_modified + 5000)
        # Меняем состояние и записываем его новое значение
        current_state.count += 1
        self.state.update(current_state)

    def on_timer(self, timestamp: int, ctx: 'KeyedProcessFunction.OnTimerContext'):
        # Запоминаем число накопленных слов
        current_state = self.state.value()
        curr_cnt = current_state.count
        # И обнуляем стейт
        current_state.count = 0
        last_modified = current_state.last_modified
        current_state.last_modified = timestamp#ctx.timer_service().current_processing_time()
        self.state.update(current_state)
        # Регистрируем таймер на очередные +5 секунд
        ctx.timer_service().register_processing_time_timer(current_state.last_modified + 5000)
        # Печатаем результат если число накопленных слов больше 0
        if curr_cnt > 0:
            yield datetime.fromtimestamp(last_modified / 1000).strftime("%H:%M:%S.%f"), current_state.key, curr_cnt
```

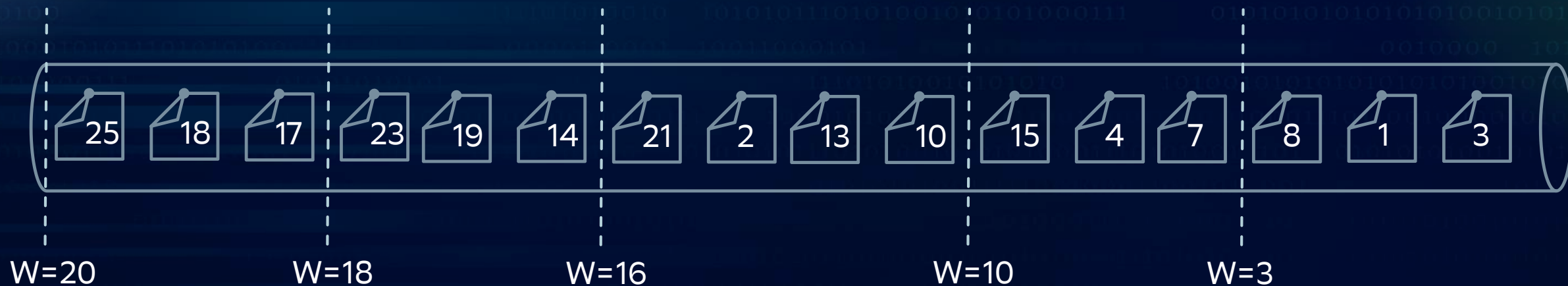
Watermarks



Watermarks



Watermarks



Simple watermark generator

Bounded out of orderness generator

Watermarks



Simple watermark generator

Bounded out of order generator

```
WatermarkStrategy.for_bounded_out_of_orderness(Duration.of_seconds(5))
```

```
.with_timestamp_assigner(MyTimestampAssigner())
```

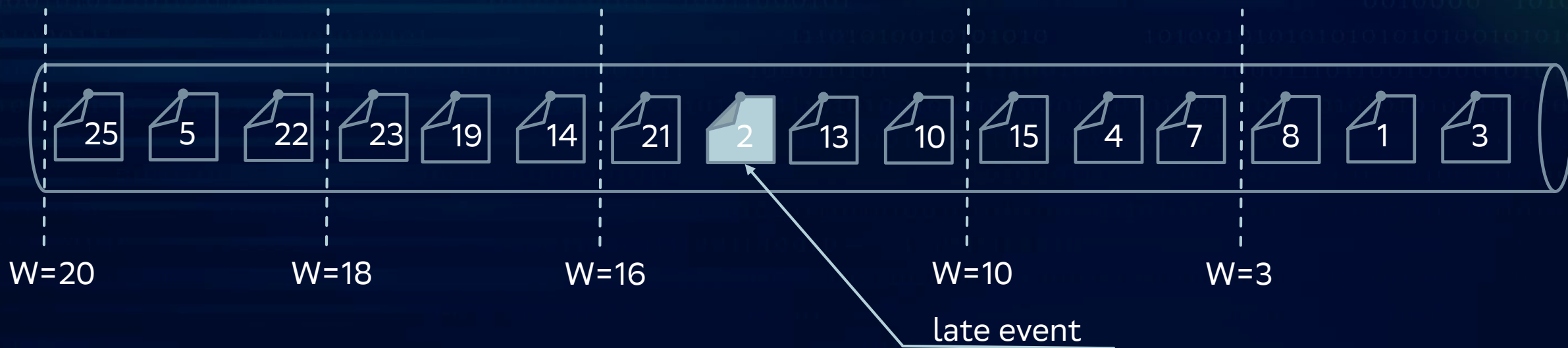
```
class MyTimestampAssigner(TimestampAssigner):  
    def extract_timestamp(self, sensor, record_timestamp):  
        return sensor[2]
```

Watermarks, example

Задача: расчет числа событий в 10-секундном окне



Lateness



event timestamp < current watermark

Приложение определяет политики по работе с late event

По умолчанию такие события отбрасываются, не учитываются в расчетах

Allowed lateness



Поток исполнения должен уметь обрабатывать такие сообщения

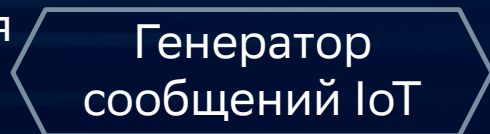
```
ds4.key_by().window().allowed_lateness(<time>)
```

Homework



Написать генератор сообщений (раз в секунду)
от IoT устройств вида:

- Тип устройства (id, идентификатор)
- Время события
- Температура
- Влажность



Эти события публикуем
в топик кафки

Разработать ddl dml скрипты
для создания и наполнения
справочника типов IoT
устройств

- Id идентификатор типа
- TypeName наименование типа



kafka

src: kafka

src: pg

Рассчитываем среднюю температуру и
медиану влажности в каждой минуте

join

window

sink: kafka

Соединяем события из
кафки со статичным
справочником из постгри

flink, работа в event time

Сохраняем полученный результат
в топик кафки в формате

- Время (hh:mm)
- Тип устройства (из pg)
- Средняя температура
- Медиана влажности

kafka

Questions



Спасибо за внимание